Course: Enterprise Real-Time Data Engineering
Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me)
Course Portal: fakhr.me/KafkaSpark
Code & Labs Repository: github.com/fakhruldeen/KafkaSpark

Session S05: Hands-On Lab Walkthrough

Confluent Schema Registry, Avro Data Contracts, Evolution & Wire Format

Lab Objectives

In this hands-on lab, you will:
1. Verify the operational health and REST endpoints of **Confluent Schema Registry**.
2. Author and register a production **Avro V1 Schema** for Nile Retail order events.
3. Inspect and decode the **5-byte Confluent Wire Format** framing raw binary events.
4. Safely evolve the schema to **V2** by adding an optional field with a backward-compatible default.
5. Attempt to register a breaking schema change (**V3**) and observe real-time enforcement and **HTTP 409 rejection**.
6. Produce and consume strongly-typed Avro events end-to-end using Python.

Step 1: Pre-Flight Environment & Registry Health Check

Navigate to the lab directory:

```
cd ~/Desktop/Work/Training/Spark/00_Lab
tools/check_stack.sh
```

Query the Confluent Schema Registry HTTP endpoint:

```
curl -s http://localhost:8081/subjects | jq .
```

Expected output: A JSON array of currently registered subjects (e.g., `[]` if fresh).

Check the global compatibility setting:

```
curl -s http://localhost:8081/config | jq .
```

Expected output:

```
{
  "compatibilityLevel": "BACKWARD"
}
```

Step 2: Register Initial Avro Schema (V1)

Inspect the initial schema contract at `schemas/order_event_v1.avsc`:

```
cat schemas/order_event_v1.avsc | jq .
```

Register this schema under the subject `orders.avro.v1-value` via the REST API:

```
SCHEMA_PAYLOAD=$(jq -c . schemas/order_event_v1.avsc | jq -R . | jq -c '{schema: .}')

curl -s -X POST http://localhost:8081/subjects/orders.avro.v1-value/versions \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  -d "${SCHEMA_PAYLOAD}" | jq .
```

Expected output:

```
{
  "id": 1
}
```

> **Explanation:** Schema Registry assigns an immutable global Schema ID (e.g., `1`). Future producers will reference this ID in the 5-byte message header.

Query the registered version metadata:

```
curl -s http://localhost:8081/subjects/orders.avro.v1-value/versions/latest | jq .
```

Step 3: Test Compatible Schema Evolution (V2)

Inspect `schemas/order_event_v2_compatible.avsc`. Notice that a new field `loyalty_tier` has been added with `"default": "STANDARD"`:

```
cat schemas/order_event_v2_compatible.avsc | jq .
```

Test compatibility against the existing subject before registering:

```
V2_PAYLOAD=$(jq -c . schemas/order_event_v2_compatible.avsc | jq -R . | jq -c '{schema: .}')

curl -s -X POST http://localhost:8081/compatibility/subjects/orders.avro.v1-value/versions/l
atest \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  -d "${V2_PAYLOAD}" | jq .
```

Expected output:

```
{
  "is_compatible": true
}
```

Register the compatible V2 schema:

```
curl -s -X POST http://localhost:8081/subjects/orders.avro.v1-value/versions \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  -d "${V2_PAYLOAD}" | jq .
```

Expected output:

```
{
  "id": 2
}
```

Verify that the subject now has two registered versions:

```
curl -s http://localhost:8081/subjects/orders.avro.v1-value/versions | jq .
```

Expected output: `[1, 2]`

Step 4: Test Breaking Schema Rejection (V3)

Inspect the breaking schema candidate `schemas/order_event_v3_breaking.avsc`. It introduces a required non-nullable field without a default value:

```
cat schemas/order_event_v3_breaking.avsc | jq .
```

Attempt to register the breaking schema:

```
V3_PAYLOAD=$(jq -c . schemas/order_event_v3_breaking.avsc | jq -R . | jq -c '{schema: .}')

curl -s -w "\nHTTP_STATUS: %{http_code}\n" -X POST \
  http://localhost:8081/subjects/orders.avro.v1-value/versions \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  -d "${V3_PAYLOAD}"
```

Expected output:

```
{"error_code":40901,"message":"Schema being registered is incompatible with an earlier schem
a for subject \"orders.avro.v1-value\""}
HTTP_STATUS: 409
```

> **Architecture Principle:** Schema Registry acts as a hard quality gate. Incompatible schema changes cannot enter the cluster, preventing poison pills at compile/registration time!

Step 5: End-to-End Avro Production & Consumption in Python

Activate the local Python environment:

```
source .venv/bin/activate
```

Produce 20 strongly-typed Avro events to `orders.avro.v1`:

```
PYTHONPATH=. python python/s05_avro.py produce --schema schemas/order_event_v1.avsc --count
20
```

Expected output:

```
produced 41 Avro events, subject orders.avro.v1-value
```

Consume and inspect the Avro events:

```
PYTHONPATH=. python python/s05_avro.py consume --group billing-audit --max-messages 5
```

Expected output:

```
{'event_id': '...', 'order_id': '...', 'customer_id': '...', 'store_id': '...', 'event_typ
e': 'OrderPlaced', 'amount': 184.2, 'event_time': '...'}
...
consumed 5 events
```

Step 6: Automated Verification Gate Execution

Run the certified test runner to validate all 5 verification gates:

```
bash ../S05_Confluent_Schema_Registry_Data_Contracts/Trainer/verify_s05.sh
```

Expected Terminal Result:

```
Running S05 Automated Schema Registry Verification Gate...
[1/5] Probing Confluent Schema Registry reachability (http://localhost:8081)... PASS (HTTP 2
00 OK)
[2/5] Registering Avro Schema V1 via REST API... PASS (Registered with Schema ID: 1)
[3/5] Testing compatible Schema evolution (V2 with default value)... PASS (Evolved to Versio
n 2 with Schema ID: 2)
[4/5] Testing breaking schema rejection (missing default value)... PASS (Correctly rejected
with HTTP 409 Incompatible Schema Conflict)
[5/5] Running Python Avro Producer against Schema Registry... PASS (Avro records produced an
d validated with Schema Registry ID)

★ PASS: S05 Lab Requirements 100% Satisfied! ★
```

🌟 Gold Stretch Challenge: Full Transitive Compatibility Validation

Distinguished Engineer Challenge: If you complete the core lab before time expires, test full transitive schema evolution:

1. Change the topic subject compatibility mode to `FULL_TRANSITIVE` via REST API:

```bash
curl -X PUT -H "Content-Type: application/vnd.schemaregistry.v1+json" --data '{"compatibility": "FULL_TRANSITIVE"}' http://localhost:8081/config/retail.orders-value
```

2. Verify that any new schema version can decode records written by version 1, and conversely, version 1 can decode records written by the new schema.

3. Attempt to remove a field that does not have a default value and confirm that Schema Registry returns HTTP 409 Conflict.